

Group 3 - Final Project Report

1. Methodology

Our project addresses three primary predictive tasks using New York City taxi trip data: fare prediction, trip duration prediction and demand forecasting for JFK Airport. Each task is formulated as a supervised learning problem, with algorithm selection guided by the nature of the target variable and the structure of the available features.

a. Machine Learning approach and Algorithms

Fare Prediction and Trip Duration Prediction

Fare prediction is approached as a regression problem. The target variable is the total fare amount for each trip, and the model leverages a variety of features, including trip distance, passenger count, pickup and drop-off locations (both as IDs and as borough/zone names), time-related variables (pickup and drop-off timestamps, day of the week, month, and season), and payment type. These features capture both spatial and temporal aspects of each trip, which are critical for accurate fare estimation.

Trip duration prediction is approached as a regression problem. The target variable is the total trip duration for each ride, measured in minutes, and the model utilizes a comprehensive set of features, including trip distance, passenger count, pickup and drop-off locations (both as location IDs), total fare amount, and engineered variables capturing trip dynamics such as fare per mile. Temporal features, including the pickup hour, day of the week, month, and time slot (morning, evening, night), were also incorporated, with cyclical encoding applied to account for the periodic nature of time-based patterns. These features collectively capture both spatial and temporal variations that influence travel time, making them critical for accurately estimating trip durations across diverse city conditions.

The machine learning algorithm we used are:

1. Linear Regression

- Chosen as a baseline due to its simplicity and interpretability.
- Helps gauge how well linear relationships can capture the variability in fare amounts.

2. Random Forest Regressor

- A robust ensemble model that handles nonlinear relationships and interactions well.
- Offers built-in feature importance, which aids in model interpretability.

3. XGBoost Regressor

- Selected for its high performance in regression tasks with structured data.
- Provides regularization to prevent overfitting and handles missing values internally.
- Its ability to model complex interactions made it a strong candidate for final deployment.

Demand Forecasting (JFK Airport)

The demand forecasting task focuses on predicting the number of taxi pickups at JFK Airport over time. This is formulated as a time series forecasting problem, where the target variable is the count of pickups (y) at specific hourly intervals (ds). The time series nature of the data requires models that can account for temporal dependencies, seasonality, and trends.

1. Facebook Prophet

- Used specifically for demand forecasting—to predict the number of trips over time.
- Prophet is well-suited for time series forecasting, particularly when the data has strong **seasonality**, **holidays**, and **trend changes**.
- Requires minimal tuning, handles missing data, and provides intuitive control over seasonality and changepoints.

2. SARIMA

- Statistical model for demand forecasting over time.
- Effective for univariate time series with clear autocorrelation structures.
- Offers high interpretability and rigorous statistical grounding. Best suited for stationary or differenced stationary series.

S No	Task	Problem Type	Target Variable	Algorithms
1	Fare Prediction	Regression	Total Fare Amount	Linear Regression
				Random Forest
				XG Boost
2	Demand Forecasting	Time Series	Pickup Count (y)	Facebook Prophet
				SARIMA

b. Data Splitting, Hyperparameter Tuning and Feature Selection

Data Splitting:

The dataset was divided into training and test sets using an 80:20 ratio:

- Training Set (80%): Used to train and validate the model.
- Test Set (20%): Held out to evaluate the model's performance on unseen data.

For Fare Prediction (Task 1), feature scaling was applied using StandardScaler, and encoding was performed before splitting to avoid any information leakage from the test set. The models were trained on spatial, temporal, and trip-based features to predict total fare amount accurately.

For Trip Duration Prediction (Task 2), a similar 80:20 random split was used to partition the dataset. Prior to splitting, continuous features such as trip distance, fare per mile, total amount, and passenger count were standardized using StandardScaler to bring all numerical variables onto a similar scale.

Additionally, feature engineering was performed to enhance the representation of temporal aspects: cyclical encoding was applied to features like pickup hour, pickup day of the week, and pickup month, allowing the models to capture the periodic nature of ride durations. One-hot encoding was used for categorical variables such as season and time slot to provide finer granularity around time-based ride behaviors. This comprehensive preprocessing ensured that the model training was robust and free from information leakage.

For Demand Forecasting at JFK Airport (Task 3), a chronological split was implemented instead of a random split. The dataset was aggregated at 3-hour intervals from the beginning of the available records through to the last recorded date (December 31, 2024). Models were trained on all historical 3-hour observations up to this point and then used to forecast pickup demand for the next 48 hours into January 2025. This respected the natural temporal sequence of the data by training models on all

historical observations up to December 31, 2024, and forecasting pickup demand for the subsequent 48 hours into January 2025.

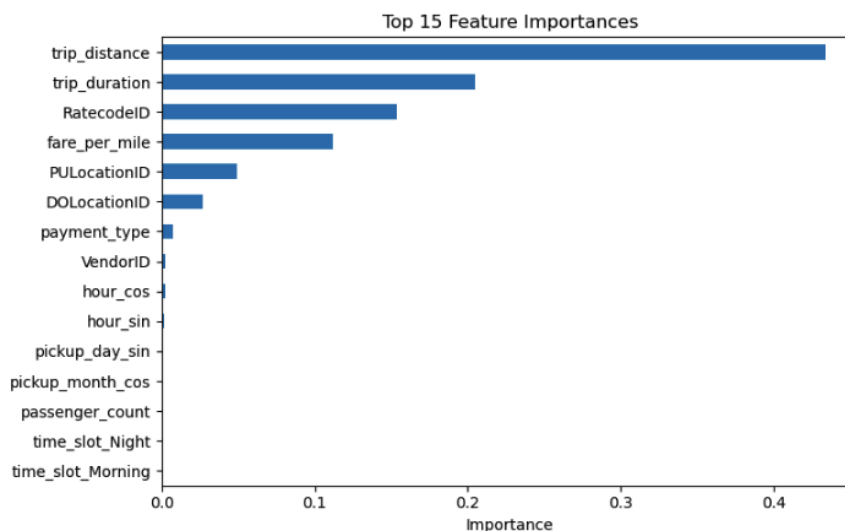
Hyperparameter Tuning

To further optimize the XGBoost model (Task 1 and 2), a smaller and focused grid search was performed over key hyperparameters including `n_estimators`, `learning_rate`, `max_depth`, `subsample`, and `colsample_bytree`. A `GridSearchCV` was configured with 3-fold cross-validation, targeting the negative RMSE as the scoring metric. The search space was carefully narrowed to balance computational efficiency and model exploration, focusing on combinations that control model complexity and regularization strength. After training, the best hyperparameter set was identified and evaluated on a hold-out test set. The optimized model achieved a low test RMSE and a high R^2 score, indicating that the tuning process significantly improved the model's predictive accuracy and generalization ability.

For Demand Forecasting at JFK Airport (Task 3), Facebook Prophet models required minimal manual tuning due to their built-in handling of seasonality, holiday effects, and changepoints. SARIMA models were tuned by selecting seasonal periods to match 24-hour daily cycles based on 3-hour aggregation intervals, without detailed grid search or manual ACF/PACF diagnostics being discussed in the original approach.

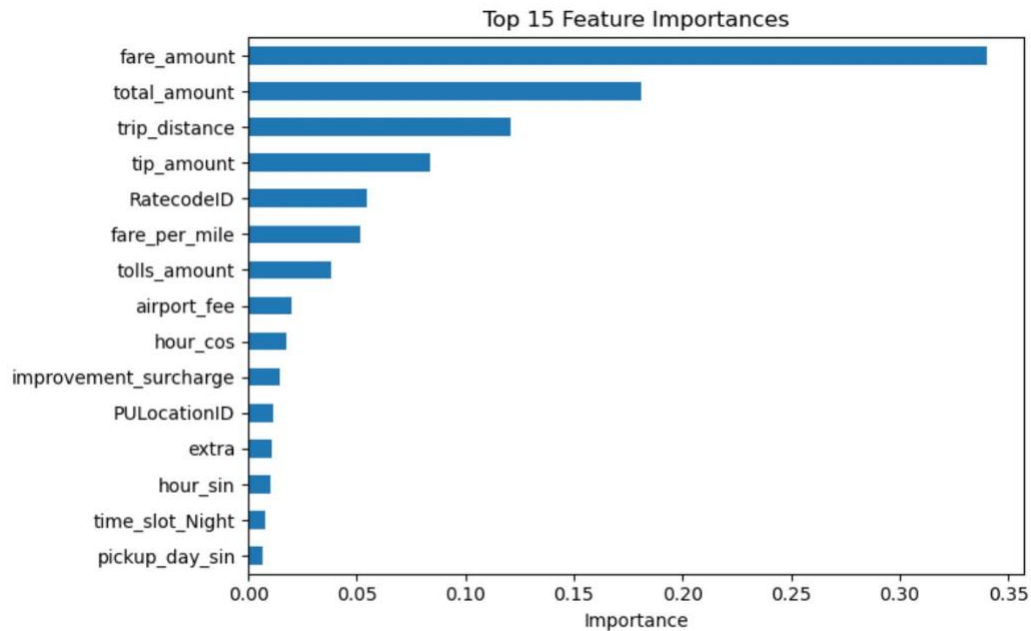
Feature Selection

For Fare Prediction, several monetary columns such as `fare_amount`, `mta_tax`, `tip_amount`, `tolls_amount`, `improvement_surcharge`, `congestion_surcharge`, `airport_fee`, and `extra` were excluded from the feature set. These fields are direct components of `total_amount` and would have led to artificially inflated model performance if included. Care was also taken to manage multicollinearity among `trip_distance`, `trip_duration`, and `trip_speed` by selectively retaining the most informative features.



For Trip Duration Prediction (Task 2), feature selection focused on including meaningful trip-level and temporal variables while carefully avoiding features that could trivially expose the target value. For instance, while trip distance and pickup/drop-off locations were included, features like directly computed trip speed were excluded. Temporal variables like pickup hour, day of the week, and month were transformed through cyclical encoding to retain their periodic structure, and categorical variables

such as season and time slot were one-hot encoded to enhance model discrimination across different ride contexts.



For Demand Forecasting at JFK Airport (Task 3), feature selection was straightforward, with only the pickup datetime (ds) and trip count (y) used. Data aggregation into 3-hour bins further streamlined the feature set while preserving essential seasonal and trend signals necessary for accurate forecasting.

2. Building Machine Learning Pipelines in AWS SageMaker

The entire machine learning workflow was implemented on AWS SageMaker AI, leveraging:

- S3 for data storage and retrieval
- SageMaker Notebooks for model development and execution

The pipeline was composed of several modular steps that handled distinct stages of the machine learning process:

a) Data Ingestion and Preprocessing

Raw NYC taxi trip data was sourced from an S3 bucket and loaded into a Pandas DataFrame using the boto3 client. The preprocessing step included handling missing values, feature scaling with StandardScaler, and feature engineering (e.g., calculating trip_speed, extracting hour, day, and month from timestamps).

b) Train-Test Split

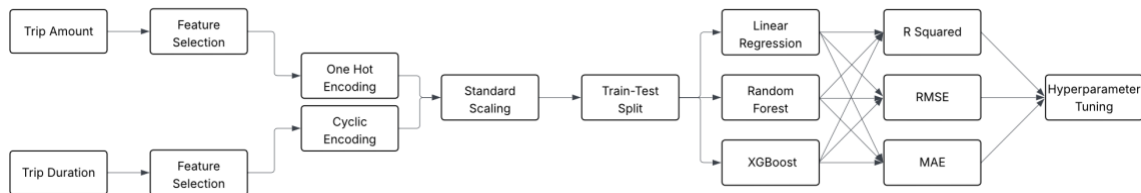
Using train_test_split from scikit-learn, the data was partitioned into training and testing sets. This ensured consistent and reproducible separation for model training and evaluation. The split ratios and random seed were parameterized for flexibility.

c) Model Training

Machine learning models including Linear Regression, Random Forest, and XGBoost were trained using their respective libraries.

d) Model Evaluation

The trained models were evaluated using metrics such as Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 Score. Additional interpretability analysis was conducted using SHAP (SHapley Additive exPlanations) to identify the most influential features in the prediction process.



Task 1:

For Fare Prediction (Task 1), the NYC taxi trip dataset was loaded from the midterm-project-01 S3 bucket. Preprocessing steps included exploratory data analysis (EDA), handling missing values, feature scaling using StandardScaler for continuous variables, and careful feature selection by removing fare-related columns that would cause data leakage. After preprocessing, the data was randomly split into training and testing sets using an 80:20 ratio. Machine learning models such as Linear Regression, Random Forest Regressor, and XGBoost Regressor were trained using scikit-learn and XGBoost libraries within SageMaker notebooks. Hyperparameter tuning for Random Forest and XGBoost was performed using GridSearchCV to optimize parameters like `n_estimators`, `max_depth`, and `min_samples_leaf`. Evaluation was based on R^2 , RMSE, and MAE scores, and SHAP analysis was conducted to interpret feature importance. Although Principal Component Analysis (PCA) was experimented with to reduce dimensionality, it was ultimately not adopted due to a noticeable decline in predictive performance.

Task 2:

For Trip Duration Prediction (Task 2), a parallel pipeline was established within SageMaker. The data was sourced from a different S3 bucket (midterm-project-01) containing a curated taxi trip dataset. Following ingestion, feature engineering steps were applied, including cyclical encoding for time-based features (pickup hour, day of the week, and month) and one-hot encoding for categorical variables such as season and time slot. Continuous features like trip distance, fare per mile, passenger count, and total amount were standardized using StandardScaler. After preprocessing, the dataset was split into training and testing subsets (80:20 split), and models including Linear Regression, Random Forest Regressor, and XGBoost Regressor were trained using SKLearnProcessor. Hyperparameter tuning was conducted through GridSearchCV to optimize model settings and achieve the best trip duration predictions. Evaluation used R^2 , RMSE, and MAE metrics to assess model accuracy.

Task 3:

For Demand Forecasting at JFK Airport (Task 3), the preprocessing pipeline involved filtering the trip records to include only rides originating from the "JFK Airport" pickup zone. Pickup timestamps were converted into datetime objects and aggregated into hourly intervals using pandas' Grouper function. To address the challenges posed by the high-frequency, high-variance nature of the hourly data—and

to better capture daily cyclical patterns—the data was subsequently down sampled into 3-hour bins, helping to smooth fluctuations and reduce computational overhead for model training.

The cleaned and aggregated data was then reformatted to match Prophet’s expected input structure, with *ds* representing timestamps and *y* representing pickup counts. Facebook Prophet models were trained within SageMaker notebooks, leveraging Prophet’s capabilities to automatically model seasonality, trend shifts, and holiday effects with minimal manual tuning. In parallel, SARIMA models were trained with a seasonal period of 8 (corresponding to the 24-hour cycle when using 3-hour bins) using a basic configuration without detailed ACF/PACF diagnostics or hyperparameter grid search. Model performance was primarily evaluated using RMSE comparisons between the two forecasting approaches.

Overall, AWS SageMaker's flexible and modular architecture, combined with S3 integration, provided a scalable and reproducible framework to conduct data ingestion, feature engineering, model training, hyperparameter tuning, and model evaluation across all three predictive tasks.

3. Model Training in AWS SageMaker

Data Ingestion from Amazon S3

For each modeling task, preprocessed datasets were uploaded to an S3 bucket named *mlc-final-dataset* and accessed programmatically using *boto3*. This allowed for centralized and secure data access across multiple training sessions. This seamless integration between Amazon S3 and SageMaker ensures efficient handling of large datasets and eliminates the need for local file storage or uploads.

Fare Prediction and Duration Prediction

Feature Selection was done as mentioned above section. Machine learning models including Linear Regression, Random Forest, and XGBoost were trained using their respective libraries. These training tasks were defined using SageMaker’s *SKLearnProcessor* and custom Python scripts, enabling the use of custom feature sets and evaluation logic. Hyperparameter tuning was conducted using *GridSearchCV* to identify optimal configurations.

For Trip Duration Prediction (Task 2), a similar model training pipeline was followed. Preprocessed datasets containing cyclical and one-hot encoded features were loaded into SageMaker notebooks, and models including Linear Regression, Random Forest Regressor, and XGBoost Regressor were trained. Feature scaling for continuous variables like trip distance and fare per mile ensured effective model learning. Hyperparameter tuning was performed separately for each algorithm using *GridSearchCV*, evaluating parameter ranges like those used in Fare Prediction. The primary goal was to minimize RMSE and maximize R^2 , ensuring the models accurately captured complex patterns affecting trip duration across different times of day and pickup/drop-off locations.

Hyperparameter Tuning and Model Optimization.

S No	Model	Hyper Parameters	Values	Grid Search Values
1	Linear Regression	Base Values		
2	Random Forest	n_estimators	50	[100, 300, 500]
		max_depth	20	[30, 10, 20]

3	XGBoost	max_features	sqrt	["sqrt", "log2"]
		min_samples_leaf	5	[2, 5, 10]
		max_samples	0.5	
		n_estimators	100	[100, 300, 500]
		learning_rate	0.1	[0.01, 0.1, 0.2]
		max_depth	6	[4, 6, 8]
		subsample	0.8	[0.6, 0.8, 1.0]
		colsample_bytree	0.8	[0.6, 0.8, 1.0]

The ranges selected for hyperparameter tuning were chosen based on a combination of domain knowledge, prior empirical evidence, and practical considerations for model complexity and generalization. For tree-based models like Random Forest and XGBoost, tuning the number of estimators and tree depth is critical because increasing these values generally improves model performance up to a point before leading to overfitting and excessive computational cost.

The selected ranges for `n_estimators` (100 to 500) and `max_depth` (4 to 30) allowed exploration of both shallow and deep trees to find a balance between underfitting and overfitting. Parameters like `min_samples_leaf`, `subsample`, and `colsample_bytree` were tuned within moderate ranges to introduce regularization and enhance model robustness by forcing trees to generalize better across different subsets of features and samples. Learning rates for XGBoost were explored between 0.01 and 0.2, reflecting the typical trade-off where lower learning rates improve performance at the cost of longer training times. Overall, the chosen tuning ranges were designed to capture the practical sweet spot between model expressiveness, training efficiency, and predictive stability.

Task – 1

Best XGB params: {'colsample_bytree': 1.0, 'learning_rate': 0.1, 'max_depth': 6, 'n_estimators': 300, 'subsample': 1.0}

Task – 2

Best XGB params: {'colsample_bytree': 0.8, 'learning_rate': 0.1, 'max_depth': 6, 'n_estimators': 300, 'subsample': 1.0}

Demand Forecasting at JFK Airport

The goal of this task is to predict the hourly and short-term demand for yellow taxi pickups at JFK Airport. Accurate demand forecasting helps optimize resource allocation, taxi dispatching, and operational planning, especially during peak periods or holidays. Feature engineering was done as mentioned in the above section.

For demand forecasting at JFK Airport, we applied two different modeling approaches: Facebook Prophet and Seasonal ARIMA (SARIMA). Prophet, developed by Facebook (Meta), is a time series forecasting tool known for its ability to model complex seasonality, trend changes, and holidays with minimal manual tuning. In this project, we fine-tuned Prophet by adding a custom 3-hour seasonal cycle ("3hr_cycle") to capture granular demand patterns linked to airport activity peaks throughout the

day. The column names are renamed to match Facebook Prophet's expected input format: ds for datetime and y for the metric to forecast.

We set `daily_seasonality=False` and `weekly_seasonality=True`, allowing the model to pick up weekly patterns while controlling daily cycles manually. Prophet's outputs included not only point forecasts but also prediction intervals and decomposable components like trend and seasonality, improving interpretability.

In parallel, we also implemented a Seasonal ARIMA (SARIMA) model, a classical statistical approach for time series forecasting. SARIMA captures four key components: Autoregression (AR), Integrated differencing (I) to remove trend, Moving Average (MA) smoothing, and Seasonality (S). We chose a seasonal period of 8 because, with 3-hourly bins, $8 \times 3 \text{ hours} = 24 \text{ hours}$ — effectively modeling daily cycles in demand. The SARIMA model was configured with `order= (1,1,1)` and `seasonal_order= (1,1,1,8)`, enabling it to handle both short-term correlations and daily seasonal patterns. This dual modeling approach using Prophet and SARIMA allowed for a comprehensive and comparative evaluation of different time series forecasting methodologies on JFK Airport taxi demand.

Comparative Model Results

Across all tasks, comparative results were evaluated using RMSE, MAE, and R^2 metrics:

Fare Prediction (Task 1):

Linear Regression provided a strong baseline but was outperformed by ensemble models. Random Forest significantly reduced errors, while XGBoost achieved the highest predictive accuracy with R^2 scores nearing 0.9964.

Model	R^2	RMSE	MAE
Linear Regression	0.89	6.52	3.76
Random Forest	0.98	2.34	1.3
XGBoost	0.99	1.23	0.69

Trip Duration Prediction (Task 2):

Linear Regression offered reasonable predictions but struggled with non-linear relationships. Random Forest and XGBoost substantially improved prediction accuracy, achieving R^2 values above 0.88 and providing robust trip duration estimations.

Model	R^2	RMSE	MAE
Linear Regression	0.83	5.45	3.13
Random Forest	0.93	3.39	1.52
XGBoost	0.93	3.28	1.46

Demand Forecasting (Task 3):

Prophet slightly outperformed SARIMA with a lower RMSE (5.36 vs 6.35), better capturing seasonal patterns in hourly taxi demand at JFK Airport.

Overall, ensemble models like Random Forest and XGBoost consistently delivered superior performance across regression tasks, while Prophet proved most effective for time series forecasting in this project.

4. Performance Evaluation Using AWS SageMaker Tools

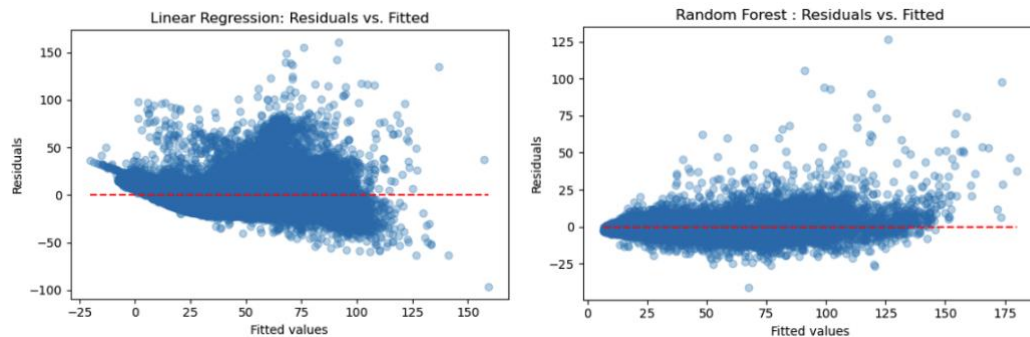
Fare Prediction:

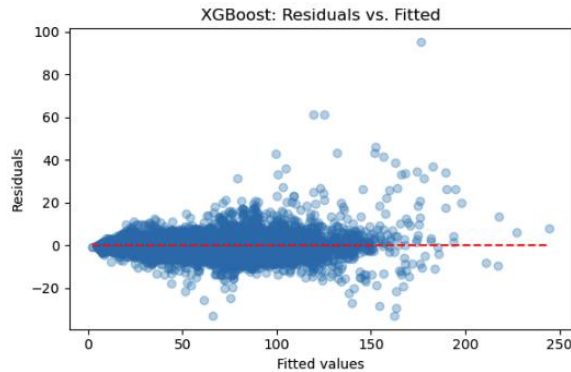
Multiple machine learning models were trained and evaluated to predict taxi fare amounts, and their performance was compared using key metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score.

The Linear Regression model achieved an R^2 score of 0.8996, indicating a reasonably good fit for the data, with an MSE of 42.56 and an MAE of 3.76. Cross-validation results showed some variability across folds, with R^2 values ranging from 0.74 to 0.91, suggesting that while Linear Regression performed decently, it struggled slightly with capturing the complex relationships in the dataset.

The Random Forest Regressor significantly outperformed Linear Regression, achieving an R^2 of 0.9871, along with a much lower MAE of 1.30 and RMSE of 2.34. This demonstrates Random Forest's ability to model non-linear relationships and feature interactions more effectively.

The XGBoost Regressor delivered the best performance among all models, with an outstanding R^2 score of 0.9964, indicating near-perfect predictive power. Its strong regularization mechanisms and ability to handle complex feature interactions likely contributed to this superior performance.





Overall, while Linear Regression provided a strong baseline, ensemble models like Random Forest and XGBoost significantly improved prediction accuracy, with **XGBoost emerging as the best model for this task**.

Trip Duration Prediction

Linear Regression:

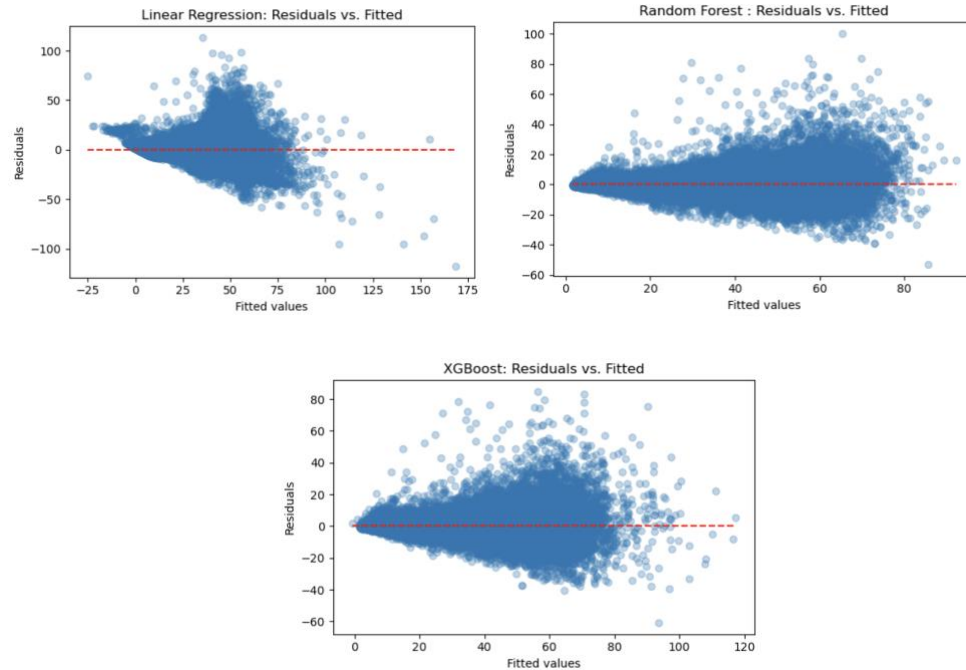
The Linear Regression model achieved an R^2 score of 0.83, suggesting it was able to explain 83% of the variance in trip duration. However, its RMSE of 5.45 and MAE of 3.13 indicate that prediction errors were relatively larger compared to more complex models. This performance suggests that while Linear Regression captured the general trend, it struggled to accurately model the non-linear relationships and interactions present in the data, limiting its precision, especially for more extreme trip durations.

Random Forest:

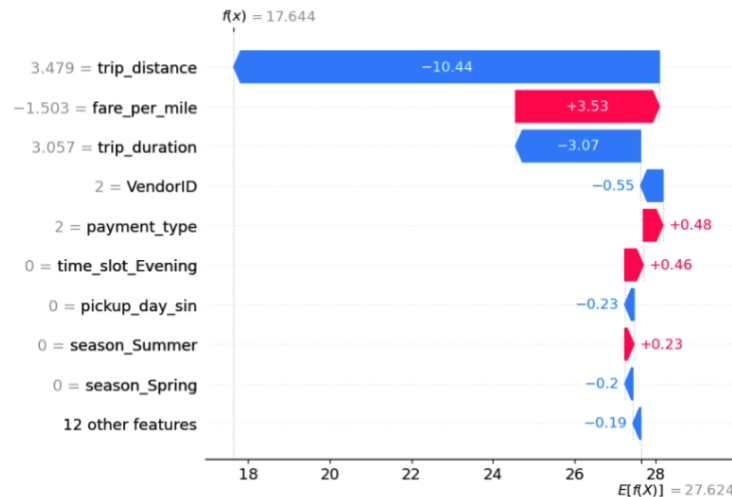
The Random Forest model significantly improved prediction performance, achieving an R^2 score of 0.93, meaning it explained 93% of the variance in trip duration. It also lowered the RMSE to 3.39 and MAE to 1.52, reflecting smaller and more consistent prediction errors. The Random Forest's ability to model non-linear patterns and feature interactions contributed to its strong performance, making it much better suited to handle the complexity of trip duration data compared to a simple linear approach.

XGBoost:

The XGBoost model delivered the best overall performance among all models, matching Random Forest with an R^2 score of 0.93 but outperforming it slightly in terms of error metrics, achieving a lower RMSE of 3.28 and MAE of 1.46. XGBoost's gradient boosting framework, combined with its built-in regularization, allowed it to minimize errors while maintaining robustness against overfitting. These results highlight that XGBoost not only captured complex patterns effectively but also provided the most precise and stable trip duration predictions.



SHAP Feature Importance Interpretation

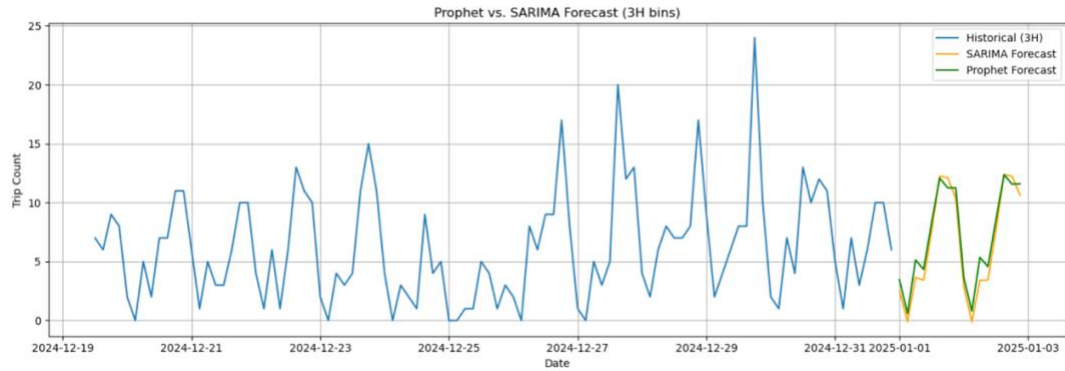


The SHAP summary plot visualizes the impact of individual features on a specific prediction, highlighting both magnitude and direction of influence. In this instance, trip_distance was the most influential feature, significantly reducing the predicted fare, followed by fare_per_mile and trip_duration, which had moderate positive and negative contributions, respectively. Features like payment_type, VendorID, and time_slot_Evening also influenced the prediction to a lesser degree. The plot emphasizes that distance-related attributes dominate fare prediction, while categorical and temporal variables contribute subtle but meaningful adjustments.

Demand Forecasting at JFK Airport

For demand forecasting at JFK Airport, both Facebook's Prophet and SARIMA models were applied to predict future taxi demand based on historical data aggregated into 3-hour intervals. The forecast

horizon was set to 16 future 3-hour bins, equivalent to a 48-hour period extending into early January 2025. Model performance was evaluated using Root Mean Squared Error (RMSE) as the primary metric. Prophet achieved an RMSE of 5.36, while SARIMA recorded a slightly higher RMSE of 6.35.

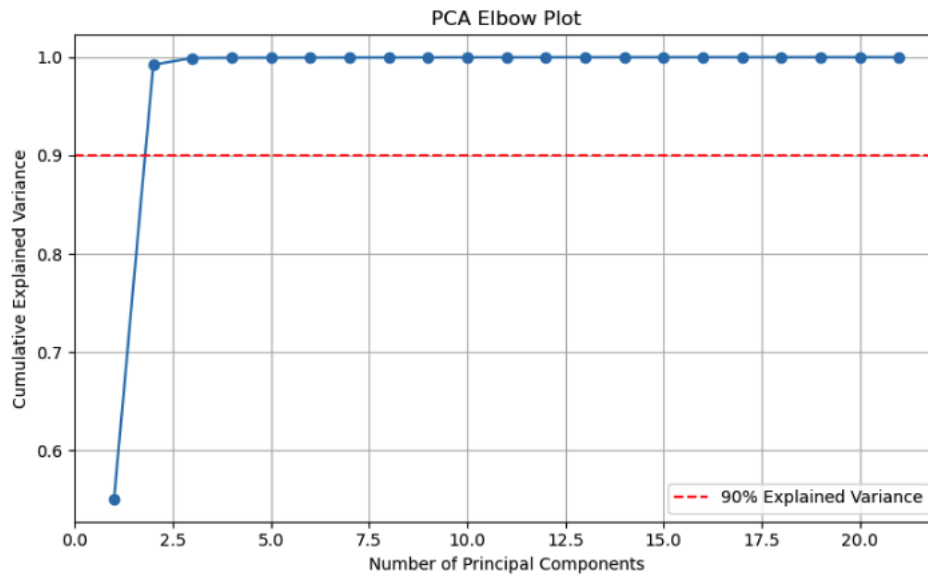


The attached graph visually compares the historical 3-hour trip counts (blue line) with the forecasted values from Prophet (green line) and SARIMA (orange line). Both models successfully captured the general structure of the demand cycle, but Prophet's forecasts more closely follow the trend and magnitude of the observed historical patterns, particularly during demand peaks and troughs. SARIMA showed strong alignment in cyclical behavior but tended to slightly underpredict or overpredict during sharp demand changes.

Overall, Prophet marginally outperformed SARIMA in this task, demonstrating better adaptability to complex, non-linear seasonality inherent in taxi demand at major transportation hubs like JFK. Its automated handling of multiple seasonalities and capacity for modeling irregular demand patterns made it particularly well-suited for this forecasting scenario.

5. Dimensionality Reduction Techniques

We applied the PCA technique for the Fare Prediction task. Although PCA was evaluated to potentially reduce training time and improve computational efficiency, results showed a significant drop in predictive performance. XGBoost achieved its best results using the full feature set (MAE: 0.544, RMSE: 0.99, R^2 : 0.997), indicating that dimensionality reduction in this case removed valuable information. Therefore, PCA was not included in the final fare prediction pipeline.



Outcome:

Original features: 21, PCA components: 16
 XGBoost with PCA:
 MAE: 1.3251673285374255
 RMSE: 2.5003738564295164
 R² Score: 0.9853794855848591

XGBoost with PCA was worse across the board:

- MAE increased by **~144%**
- RMSE increased by **~152%**
- R² dropped from **0.997 → 0.985**
- Despite reducing dimensionality from 21 → 16, **it lost predictive power.**

Although PCA was evaluated to potentially reduce training time and improve computational efficiency, results showed a significant drop in predictive performance. XGBoost achieved its best results using the full feature set (MAE: 0.544, RMSE: 0.99, R²: 0.997), indicating that dimensionality reduction in this case removed valuable information. Therefore, PCA was not included in the final fare prediction pipeline.

6. Results and Discussion

Key Findings: Fare Prediction and Trip Duration Prediction

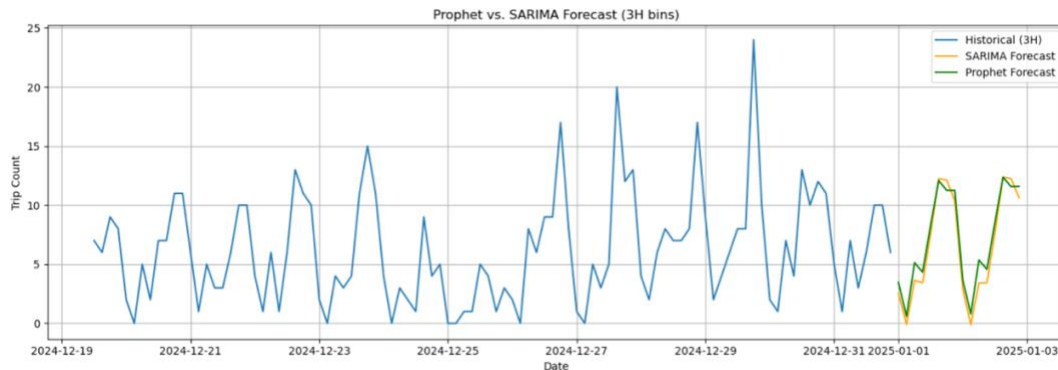
Ensemble models dominate: Both Random Forest and XGBoost consistently delivered better predictive accuracy and lower errors compared to Linear Regression across both fare and trip duration prediction tasks.

XGBoost leads: While Random Forest closed the gap considerably, XGBoost consistently offered the best balance of accuracy, robustness, and error minimization.

Linear models are limited: Though easy to interpret and quick to train, Linear Regression struggled with real-world taxi data due to its linearity assumptions and inability to capture complex feature interactions.

Feature engineering matters: Careful handling of features like distance, duration, and fare per mile greatly influenced the success of the models, especially for ensemble-based approaches.

Key Findings: Demand Forecasting



This visualization aims to predict taxi demand at JFK Airport by forecasting the number of yellow cab pickups in each 3-hour interval. Accurate forecasts support efficient resource allocation, reduced passenger wait times, and optimized traffic management around the airport. Each point on the Y-axis represents the trip count within a 3-hour window. The forecast horizon covers 48 hours beyond the last recorded data point on December 31, 2024, at 21:00, extending from January 1, 2025, 00:00 to January 2, 2025, 21:00, resulting in 16 forecasted time steps. The blue line in the chart shows historical trip counts, while the green and orange lines represent forecasts by Prophet and SARIMA models, respectively. Both models capture demand fluctuations, but Prophet's forecasts are smoother, more adaptive, and better aligned with demand spikes, achieving a lower RMSE of 5.36 compared to SARIMA's RMSE of 6.35. Prophet's design, which explicitly models seasonality, trend changes, and holiday effects, makes it particularly suited to environments like JFK, where demand is driven by complex seasonal patterns. In contrast, SARIMA, although robust for stationary series, struggles with multiple seasonalities and abrupt changes unless heavily tuned, explaining its slightly inferior performance here.

Strengths:

A key strength of this approach is its ability to effectively model complex seasonal patterns and demand spikes without requiring extensive manual tuning. Prophet's flexibility in automatically detecting holidays, trend shifts, and recurring patterns allows it to adapt well to dynamic environments like airports, where multiple overlapping factors influence taxi demand. Additionally, SARIMA's strong baseline performance demonstrates the robustness of traditional time series methods, providing a valuable point of comparison and validation. The use of a 3-hour aggregation strategy also ensures a practical balance between forecast resolution and computational efficiency, enabling timely model training and deployment. Moreover, leveraging AWS cloud services greatly facilitated the handling and import of the large-scale NYC Yellow Taxi dataset, which would have been extremely challenging to process locally due to its volume.

Challenges and Limitations:

However, a key limitation encountered was the computational intensity of ensemble methods, particularly during hyperparameter tuning, which required substantial time and resources. Linear models, while efficient, struggled to capture the complexity of real-world taxi data, especially for extreme fare and trip duration values. Another challenge was ensuring model robustness across varying data distributions and managing heteroscedasticity, especially in simpler models.

One major challenge encountered during model development was the computational intensity associated with using hourly data granularity. Forecasting at an hourly level significantly increased training time and memory consumption, making it impractical for rapid experimentation and model tuning. As a result, the dataset was aggregated into 3-hour bins to balance computational feasibility with forecast accuracy. However, this aggregation introduces a limitation: fine-grained demand variations within each 3-hour window are smoothed out, potentially obscuring short-term demand surges that are critical for operational decision-making at an airport.

Potential Improvements and Areas for Future Work:

While the current models achieved strong results, several areas remain for potential improvement. Incorporating external data sources such as real-time traffic information, weather conditions, or special event schedules could further improve predictive accuracy for both fare and trip duration. To address the computational demands of ensemble models, techniques like model pruning, feature selection optimization, or distributed training across multiple compute nodes could be explored.

Future work can focus on enhancing forecast granularity by leveraging more powerful compute resources or adopting lightweight, scalable models like NeuralProphet or Temporal Fusion Transformers that can handle high-frequency data more efficiently. Additionally, integrating external features such as real-time flight schedules, weather conditions, and public transportation availability could significantly improve forecast accuracy by capturing external demand drivers.

7. Conclusion and Future Work

This project successfully implemented a comprehensive machine learning pipeline for three key predictive tasks related to New York City taxi data: fare prediction, trip duration prediction, and demand forecasting at JFK Airport. Using AWS SageMaker and scalable cloud resources, we developed and evaluated multiple models including Linear Regression, Random Forest, XGBoost, Facebook Prophet, and SARIMA.

Across tasks, ensemble models (Random Forest and XGBoost) consistently outperformed simpler baselines, achieving high predictive accuracy with XGBoost delivering near-perfect results for both fare and trip duration prediction. For time series forecasting, Facebook Prophet slightly outperformed SARIMA, demonstrating stronger adaptability to complex seasonal and trend variations inherent in airport demand patterns.

The use of advanced preprocessing techniques such as cyclical encoding, hyperparameter optimization with GridSearchCV, and model explainability using SHAP greatly enhanced model performance and interpretability. Overall, the project demonstrates that combining robust machine learning methods with scalable cloud infrastructure can deliver accurate, reliable, and interpretable predictive insights in transportation analytics.

While the current models achieved strong results, future work could focus on improving computational efficiency, especially for ensemble methods, by leveraging model compression techniques or

distributed training. Enhancing the feature set with external data sources such as weather conditions, flight arrival schedules, or real-time traffic data could further boost predictive performance. For demand forecasting, experimenting with more advanced deep learning models like LSTM or Temporal Fusion Transformers (TFT) could provide even better handling of long-term dependencies and multiple seasonalities. Continuous model retraining with fresh data and monitoring for concept drift will also be essential to maintain high forecasting accuracy in dynamic operational environments.